

Role Labels or Extra Calls? A Controlled Study of Quality and Cost in LLM Code Generation

ANONYMOUS AUTHOR(S)

LLM code-generation workflows often combine named roles with multiple model calls, making it unclear which design choice changes quality and resource use. We disentangle these choices in a crossed experiment: neutral versus planner–implementer–reviewer labels, and one versus three calls, with identical operations and full task context; a direct solver is a fifth condition. GPT-5.6 Luna receives 15,000 assignments on 1,000 BigCodeBench tasks, with three fresh repeats per condition. Of 14,961 completed candidates, 985 tasks pass reference controls. Role labels change success by -0.75 percentage points in one call and $+1.35$ points in three calls; neither comparison rejects after Holm correction. Three calls reduce paired success by 4.17 points under neutral labels and 2.13 points under role labels, while raising API-equivalent valuation by factors of 2.67 and 2.78. An earlier 1,000-assignment GPT-5.4-mini study on the reused 200 task IDs has the same sign for all four factorial contrasts, although none rejects after correction. A prespecified source-dependence audit and a separate analysis of 800 newly sampled task IDs preserve the negative topology result. Exploratory direct-solver differences from the one-call workflows are small with intervals crossing zero; its observed paired success is higher than both three-call workflows. A separate 2,000-block comparison with an adapted Self-Collaboration controller gives lower observed-pair success and approximately three times the valuation, but infrastructure missingness leaves its full-population quality difference unidentified. These results provide evidence for a boundary case: for the tested models and full-context function-generation workflow, role labels alone show no robust benefit, while extra call boundaries increase cost and reduce executable success.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; *Software testing and debugging*.

Additional Key Words and Phrases: LLM code generation, orchestrated prompting, role prompting, factorial experiment, executable evaluation, resource accounting

ACM Reference Format:

Anonymous Author(s). 2027. Role Labels or Extra Calls? A Controlled Study of Quality and Cost in LLM Code Generation. In *ACM International Conference on the Foundations of Software Engineering (FSE 2027)*, 12–16 July 2027, Shenzhen, China. ACM, New York, NY, USA, 15 pages.

1 Introduction

Code-generation workflows increasingly assign planning, implementation, and review to named LLM roles. Systems such as Self-Collaboration, ChatDev, MetaGPT, AgentCoder, and PairCoder combine role specialization with interaction, generated tests, tool use, or repeated refinement [5, 7, 9, 19, 26]. Their end-to-end results do not identify whether a gain comes from the role names, the operations, additional calls, feedback, or extra computation. That distinction matters in software engineering: each added call can repeat a large repository context, expose an unverified intermediate response, and increase cost. We evaluate role-labelled sequential LLM workflows, not independent agents; multi-agent systems motivate the design space but are not the construct tested by the primary factorial experiment.

We study a deliberately narrow workflow. The model is asked to analyze requirements and edge cases, implement from that analysis, and review and revise the program. Correctness is then measured by executing the original benchmark tests in a pinned offline environment; the model’s textual assertion is never an outcome. We cross two factors while keeping operation sentences and supplied context fixed: neutral versus role labels, and one versus three fresh calls. A direct one-call solver supplies the lower-complexity baseline. The design therefore estimates component effects rather than treating a branded architecture as a single indivisible treatment.

FSE 2027, Shenzhen, China
2027.

The study asks three research questions. **RQ1:** Do role labels change executable success when the requested operations and call structure are held fixed? **RQ2:** Does splitting those operations across three full-context calls change success and measured resources relative to one call? **RQ3:** How do the staged workflows compare with a direct solver and with a published analyst–developer–tester controller? The first two questions form the prespecified factorial analysis. Direct-baseline comparisons, the role-by-topology interaction, and subgroup checks are labelled exploratory. The external-controller study has its own prespecified family and missingness analysis.

The labels planner, implementer, and reviewer denote prompt segments; they do not demonstrate separate latent agents, independent memories, or independent optimization. The experiment uses 1,000 BigCodeBench tasks, three fresh repeats per condition, and all 14,961 completed candidates. It also preserves the design history: 200 task IDs came from an earlier study and 800 were newly sampled, but every response in the expanded study was generated afresh. We therefore report the 200/800 sensitivity separately and do not call the full sample an independent 1,000-task replication.

The paper contributes (1) a crossed operational design separating role labels from call boundaries; (2) task-paired executable estimates of quality and resource consequences with prespecified missingness and dependence analyses; and (3) external-validity comparisons with a second model alias, a direct solver, and an adapted Self-Collaboration controller. The direct and interaction comparisons are exploratory, and the study does not claim a general multi-agent effect. The main finding is conditional and negative: in this full-context setting, role labels do not yield a robust quality benefit, while three-call execution costs substantially more and succeeds less often than the matched one-call workflow.

2 Related Work and Scope

Self-Collaboration decomposes code generation into analyst, developer, and tester roles, and reports role and interaction-count ablations [5]. ChatDev and MetaGPT coordinate specialized roles across broader software-development processes [7, 19]. AgentCoder closes a generated-test feedback loop among programmer, test designer, and executor; the ASE PairCoder couples navigator and driver roles with multi-plan exploration and execution feedback [9, 26]. MapCoder combines retrieval, planning, coding, and debugging roles [10]. AdaCoder evaluates four frameworks across 12 open-source models before adding adaptive planning and feedback [29]; the ACL PairCoder evaluates a lighter two-agent scheme across 13 models and explicitly measures tokens [3]. A systematic review identifies 71 primary studies of LLM multi-agent systems for software engineering and highlights the unresolved need to isolate agent capabilities from collaboration effects [6].

Recent controlled comparisons examine homogeneous workflows reproduced by one model, adaptive topology selection, and quality–cost trade-offs [8, 11, 24]. These studies vary complete architectures, allocate different compute, or use executable feedback. Our narrower contribution is a crossed intervention that changes lexical role labels and call boundaries while holding operation sentences and supplied context fixed. Self-Collaboration is the closest workflow target: its published study varies role configurations and interaction counts, while our factorial uses repeated native outcomes, complete-context accounting, and explicit missingness. Our separate SCC comparison ports its analyst–developer–tester controller to the same model, tasks, and external evaluator, while disclosing transport changes. Table 1 makes these distinctions explicit. INoT motivates compact internal dialogue and role-like reasoning instructions [21]; that intervention changes the reasoning procedure and is therefore kept outside the primary factorial family.

Persona prompting studies likewise distinguish a named persona from a demonstrated performance benefit [12, 27]. Self-Refine and Reflexion use feedback and revision [14, 20]. Execution-guided methods such as CodeT and LDB use generated tests or runtime traces to select or repair

Table 1. Closest code-generation workflow studies. “Feedback” means generated tests or runtime outcomes returned during generation. None besides this work isolates lexical role labels while holding operation sentences fixed.

Work	Primary variation	Feedback	Cost	Direct baseline
SCC [5]	Role configurations and turns	Yes	No	Yes
MapCoder [10]	Four-role repair pipeline	Yes	No	Yes
AdaCoder [29]	Adaptive planning across models	Yes	Yes	Yes
PairCoder [3]	Two-agent switching across models	Yes	Yes	Yes
OneFlow [24]	Multi- versus single-model execution	Varies	Yes	Yes
DATS [8]	Five task-adaptive topologies	Varies	Yes	Yes
This work	Labels $\times$ one/three calls	No	Yes	Yes

programs [2, 28]. These interventions can improve code generation but bundle topology with executable feedback. Our primary matrix withholds all executable feedback until generation ends, so it estimates labels and call boundaries rather than execution-guided repair.

Additional-call comparisons can confound call count with computation, repeated context, and exposure of intermediate responses. Matched-thinking-budget work emphasizes that these are distinct interventions [22]. Cost-aware agent evaluation likewise argues for joint accuracy–cost reporting and strong simple baselines [11]. We therefore do not call topology a pure call-count effect and do not impose a post hoc matched output budget. Instead, we report input, cached input, output, and API-equivalent valuation as consequences of the implemented workflows. Agentless and SWE-agent add localization, repository access, or tool interaction [23, 25]; those systems address a broader repository setting than the fixed-context function-generation matrix here. BigCodeBench supplies diverse library-dependent tasks with executable tests [30]. Native evaluation improves auditability while retaining the benchmark’s task-test limitations.

3 Workflow and Estimands

An engineering task is $(x, C_0, \mathcal{V})$, where x is the specification, C_0 is the complete supplied context, and $\mathcal{V}$ is an external verification procedure. A protocol a produces a candidate program y and a verification record. In the primary study, $r = 1, 2, 3$ indexes the three requests for task t . When its outcome is available, $Y_{tr}(a) \in \{0, 1\}$ records whether the candidate passes the original tests in a valid environment. Missing outcomes remain undefined. For tasks with all three outcomes, $\bar{Y}_t(a) = \frac{1}{3} \sum_{r=1}^3 Y_{tr}(a)$ is the per-generation success mean. Each primary contrast averages differences between these means over eligible tasks with all six required outcomes. Thus its estimand is conditional on a complete pair, with equal task weights. The engineering objective is to raise verified success while controlling expected token use and generation cost. We report quality and resources separately because no externally justified joint utility or quality-loss margin is available.

In the single-call core, one model call receives x , C_0 , and three ordered operations: analyze requirements and edge cases; implement using the plan; review and revise against the requirements. It returns exactly one fenced Python program. The hidden reasoning trace is unavailable and is not treated as an observed plan or critique score. In the multi-call version, the same three operations are assigned to three fresh calls. Call j receives the task, unchanged context, and every complete preceding response. The final response supplies the candidate. No benchmark test, reference solution, or evaluator feedback is returned to the model.

The five conditions are D (direct implementation), SN (neutral stage labels in one call), SR (role labels in one call), MN (neutral labels across three calls), and MR (role labels across three calls). All structured conditions use identical operation sentences and final-output contract. The label intervention substitutes stage names with planner, implementer, and reviewer. Thus the

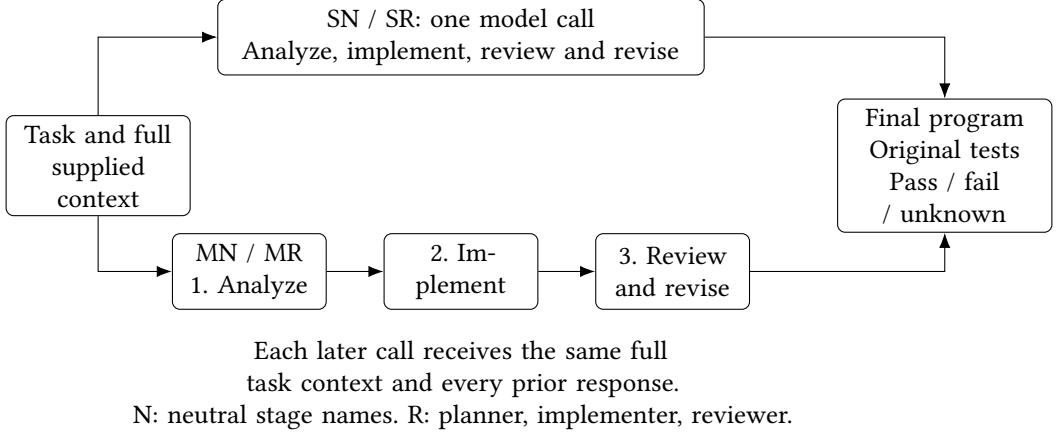


Fig. 1. The crossed intervention. The direct baseline D has one call without staged instructions. Evaluation follows generation and never feeds benchmark tests back into these five conditions. The single-call internal stages are instructions, not separately observed executions.

topology contrast includes repeated full-context transmission and intermediate-response exposure; it is not a mechanical count-only effect.

The observable state after call j is $\sigma_j = (x, C_0, a, j, \mathcal{H}_j)$, where $\mathcal{H}_j$ contains complete preceding responses. Generation runs sequentially until the assigned number of calls is complete, or a call is incomplete. Resource availability is checked separately from candidate quality. Incomplete assignments remain unavailable and are not retried automatically. A completed response with zero or multiple recognized Python fences yields an empty observed candidate and a format-error flag. The evaluator then records pass, failure, or unavailable. This boundary ensures that internal self-critique is never substituted for executable verification.

The resource motivation is conditional. Let L be the token length of the task and context, H_j the instruction overhead, E_j the exposed answer length, and O_j all output tokens of call j . Hidden reasoning may contribute to O_j but is not forwarded in E_j . In an additive block model, a three-call sequence uses

$$T_3 = 3L + \sum_{j=1}^3 (H_j + O_j) + 2E_1 + E_2.$$

A single call uses $T_1 = L + H_s + O_s$. Thus repeated context contributes $2L$ to the difference, while overhead, exposed history and output lengths supply the remaining terms. Their sum need not be positive. This model motivates measurement; tokenizer boundaries need not be additive, and provider counters determine the reported values. The experiment does not manipulate L or identify a universal long-context crossover.

3.1 Prompt construction

The common execution instruction is: “Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available.” The three exact operation sentences are:

- (1) Analyze requirements and produce an implementation plan, including edge cases.
- (2) Implement the requested program or repository change using the plan.

- (3) Review correctness against the requirements, fix problems and provide the final implementation.

Each sentence has a prefix and colon: either stage 1, stage 2, stage 3 or planner, implementer, reviewer. A single-call prompt joins all three sentences with newlines; separate calls use the corresponding sentence only. D uses “Implement the requested program.” All final calls add: “Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available.”

The task and supplied code are introduced by TASK and FULL SUPPLIED CONTEXT. Every earlier response is appended under PREVIOUS STAGE j (COMPLETE OUTPUT), with one-based j . Thus later calls see all earlier text without truncation. Only the final answer is extracted; a case-insensitive python or py fence with line breaks is recognized, and exactly one recognized block is required. The extracted block has its boundary whitespace stripped. Other language tags are ignored. Malformed completed answers yield empty candidates; interrupted answers do not yield invented programs.

4 Experimental Design

4.1 Factorial intervention and estimands

The main study uses BigCodeBench v0.1.4 in its `instruct_full` mode [30]. The source pool contains 1,140 tasks. IDs are sorted as strings and shuffled with `Python 3.11 random.Random(20260908)`; the first 40 form the development set. Tasks /0–/7 were public examples used while checking the extraction and evaluation path, so they were excluded before model generation to avoid evaluating on exposed development material. The initial factorial takes 200 further IDs, and the next 80 form a separate layout set. The expanded allocation retains the 200 factorial IDs and adds 800 from the remaining permutation, excluding development and layout IDs. The expanded study reuses the 200 IDs from the earlier factorial study and adds 800 IDs; earlier programs are not pooled. Three repeat labels correspond to fresh requests, not provider sampling seeds. Every task receives all five conditions. The design therefore supplies 15,000 assignments and represents at most 1,000 sampled task units, with each task contributing repeated observations.

For each task, the supplied model input contains the original instruction and code context, including starter code. Tests and canonical solutions are withheld. The final output contract requires exactly one fenced Python block. Before any Luna benchmark response, an availability amendment replaced an unsuccessful GPT-5.4-mini launch, which produced no candidate responses. The Luna study generated every assignment afresh. All requests use GPT-5.6 Luna with medium reasoning through Codex CLI 0.153.4 [16, 18], a 600-second per-call deadline, and a clean configuration with tools, browsing, delegation, and test execution disabled. Inputs above 65,536 UTF-8 bytes are rejected rather than shortened. No temperature or provider seed is claimed.

The primary estimands are four paired task-level differences in three-repeat means:

$$\Delta_{R,1} = \mathbb{E}_t[\bar{Y}_t(\text{SR}) - \bar{Y}_t(\text{SN})], \quad \Delta_{R,3} = \mathbb{E}_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{MN})], \quad (1)$$

$$\Delta_{C,N} = \mathbb{E}_t[\bar{Y}_t(\text{MN}) - \bar{Y}_t(\text{SN})], \quad \Delta_{C,R} = \mathbb{E}_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{SR})]. \quad (2)$$

For each contrast, all three repeat outcomes in both conditions must be available; the difference of the two three-repeat means then receives equal task weight. The four two-sided one-sample Student t -tests on these complete-pair differences form one Holm family at $\alpha = 0.05$. Bootstrap intervals resample whole tasks 10,000 times with PCG64 seed 20260909 and are descriptive. We do not interpret failure to reject as equivalence or non-inferiority.

The paired construction is chosen because each condition is assigned to the same task and repeat labels. A task enters a contrast only when its three outcomes are available in both conditions, so the comparison uses paired per-generation means rather than marginal rates with different task mixes. The four contrasts have different eligible task sets, whose sizes are reported with the estimates. Pairing does not eliminate dependence among related benchmark tasks or provider-level correlation across requests.

A source-dependence sensitivity groups related tasks by the union of exact normalized code and instruction-token similarity at 0.70, and separately at instruction-only thresholds 0.50, 0.70 and 0.90. Whole groups are resampled, retaining task weights, with PCG64 seed 20260911 and 10,000 draws. This checks a limited form of benchmark dependence [1, 13]; it cannot detect training contamination. The union partition has 992 groups among the 1,000 tasks. Its four quality intervals are reported beside the corresponding contrasts in Table 3.

For descriptive missingness bounds, each unavailable binary endpoint is completed once in the unfavourable direction and once in the favourable direction, preserving the assigned denominator. The resulting interval answers how far the observed rate or paired difference could move under the recorded missing rows. It is separate from the task bootstrap, which represents sampling variation under the observed eligible outcomes. Neither calculation supplies a causal adjustment for transport failures, deadlines, or a task being more difficult than another. The study reports these quantities side by side so that a narrow sampling interval is not mistaken for proof that unavailable responses were random.

The assignment schedule was fixed before generation and controls were frozen before model quality inspection. Python randomization with order seed 20260908 permuted the five conditions within each task-repeat block and the block order. Actual completion order also depends on concurrency and interruptions; this does not control provider drift. Workers process disjoint task shards with bounded concurrency, but worker order does not determine the condition or task assignment. A continuation amendment could start only never-submitted cells after interruptions; a cell with any started call remained permanently excluded from continuation. This preserves the distinction between the protocol’s planned population and the rows that became observed, and prevents a failed response from triggering outcome-dependent replacement.

4.2 Native controls and missingness

Reference controls concatenate the supplied code context and canonical solution. Incorrect controls append a function returning None. A task is quality-eligible only when the reference passes and the incorrect control fails. Native evaluation runs in an offline container with two CPUs, a 3-GB memory cap, and the pinned BigCodeBench dependencies. For eligible tasks, a native pass is one and a native failure or test timeout is zero. Missing generation, an incomplete evaluator run, and an invalid control remain unavailable. All 15,000 assigned rows, including the 15 control-ineligible tasks and incomplete candidates, remain in completion and resource accounting.

Generation produced 14,961 complete candidates; 39 submitted assignments remained incomplete. Every complete candidate entered native evaluation. The reference gate leaves 985 possible quality-analysis tasks, while 264 assigned endpoints are unavailable because of generation or control status. Unknown endpoints are not imputed as failures or successes. Assignment-denominator bounds are reported where useful to show the extreme effect of missing binary outcomes; they are not confidence intervals and do not correct selection bias.

The retained endpoint taxonomy separates generation from test failure. Across D, SR, SN, MR, and MN respectively, incomplete generations number 4, 3, 2, 13, and 17; native timeouts number 9, 6, 9, 7, and 7. No completed primary response fails the frozen one-block extraction rule. Ordinary native failures account for the remaining unsuccessful evaluated programs. These records identify

where failure occurred, but the design does not expose a counterfactual revision trace, so it cannot attribute a three-call failure to a particular intermediate message.

The control gate confirms that the canonical solution runs and that a deliberately incorrect candidate fails before model outcomes are interpreted. A task failing either control remains in assignment and resource summaries but leaves the quality estimand. This prevents an unavailable or insensitive test from being reclassified after seeing responses. Extraction is equally explicit: a completed answer must contain exactly one recognized fenced Python block. A missing, multiply fenced, or differently tagged answer yields an empty observed candidate and a format flag, while a call that never completes remains unavailable. Thus malformed responses contribute measured generation cost, whereas absent responses may have only partial or unknown counters.

The evaluator runs each candidate against the unchanged original tests in the pinned local environment. Model-call and evaluator timeouts are separate stopping rules. Native pass, ordinary failure, and native timeout are retained as distinct records. A model-generated explanation, printed example, or internal review cannot replace an executable pass. This separation is the practical meaning of external verification in the protocol.

4.3 Use of AI systems in the research process

The evaluated programs are AI-generated data, so model use is part of the method rather than writing assistance. GPT-5.6 Luna at medium reasoning generated all 15,000 primary assignments on 11–13 September 2026 and all 6,000 selected SCC-comparison assignments on 13 September through Codex CLI/runtime 0.153.4. GPT-5.4 mini generated the earlier 200-task corroborating study on 8 September 2026 [17]. The frozen records retain the exact prompts, model aliases, request order, context bytes, timestamps, status events, extraction outcomes, token counters, and evaluator commands. Provider sampling seeds and immutable weight identifiers were unavailable; no unsupported value is inferred for them. Models had no tools, browsing, repository access, benchmark tests, reference solutions, or evaluator feedback during the primary generation.

OpenAI Codex with GPT-6, accessed through the desktop and command-line service on 8–20 September 2026, assisted protocol drafting, experiment implementation, analysis and packaging code, consistency checks, literature discovery, and language revision; the service did not expose an immutable weight revision. It produced draft protocol prose and portions of the executable code. The authors inspected every retained change, ran the unit and integration tests, replayed analyses from frozen ledgers, compared generated tables with source summaries, checked cited claims against primary records, and manually reviewed the rendered PDF. Every reported number is produced from retained machine-readable records by versioned scripts or independently recomputed during the audit; model prose is not treated as evidence. The authors selected the design, froze amendments before the corresponding outcome inspection, inspected failures, and remain responsible for the submission. AI-assisted internal reviews are author-side quality control and are not represented as external peer review.

5 Results

5.1 Quality in the 1,000-task study

The observed success rates range from 50.07% to 54.59% across the five conditions. The role-minus-neutral contrast is -0.75 percentage points in one call (95% interval $[-1.97, 0.44]$) and $+1.35$ points in three calls (95% interval $[0.17, 2.56]$). The positive three-call interval excludes zero descriptively, but its Holm-adjusted $p = 0.0599$ does not pass the prespecified familywise threshold. Neither role-label comparison supports a primary quality rejection.

Table 2. Luna factorial study: assignment and observation counts.

Condition	Assigned	Generated	Observed	Passes	Success (%)
D	3000	2996	2951	1611	54.59
SN	3000	2998	2953	1604	54.32
SR	3000	2997	2952	1581	53.56
MN	3000	2983	2938	1471	50.07
MR	3000	2987	2942	1514	51.46

Observed means eligible native quality endpoints, not distinct tasks. Success divides passes by observed repeat outcomes. Each condition has three assigned repeats per task.

Table 3. Luna factorial quality contrasts. Differences and bounds are percentage points.

Contrast	Tasks	Difference [95% interval]	Holm p	Assigned bounds
SR – SN	982	−0.75 [−1.97, 0.44]	0.2357	[−2.33, 0.83]
MR – MN	964	+1.35 [0.17, 2.56]	0.0599	[−0.63, 3.37]
MN – SN	968	−4.17 [−5.65, −2.75]	1.34×10^{-7}	[−6.00, −2.37]
MR – SR	971	−2.13 [−3.43, −0.82]	0.0044	[−3.83, −0.30]

Tasks require three observed repeats in each compared condition. Assigned bounds use all 1,000 tasks and every assigned repeat; they are not confidence intervals. Source-group intervals for the four rows are [−1.99, 0.47], [0.14, 2.60], [−5.69, −2.72], and [−3.44, −0.79].

Call structure shows the larger and consistent change. On complete eligible task pairs, three calls have lower success by 4.17 points for neutral instructions and 2.13 points for role-labelled instructions, and both contrasts reject after Holm adjustment. Their full-assignment identification bounds remain negative, so the direction is not produced solely by assigning favourable values to missing endpoints. The descriptive role-by-call interaction is +2.08 points with interval [0.42, 3.74] on the common 963-task subset; it is outside the four-test family. These are per-generation outcomes, not best-of-three selection results.

5.2 Direct baseline and sampling-history audits

The estimated direct-solver differences from the one-call workflows are small: D–SN is +0.27 points and D–SR is +0.95 points, with both intervals crossing zero (Table 4). These comparisons do not establish equivalence. D has 3.20–4.59 points higher observed paired success than the three-call workflows. Because the direct contrasts were not part of the four prespecified factorial tests, we report them as exploratory estimates rather than extend the confirmatory family after observing results. They answer RQ3 without providing evidence that staged prompting is necessary for the observed quality level.

The sampling-history split reaches the same central conclusion (Table 5). The neutral three-call effect is −4.17 points in both the 200 reused task IDs and the 800 newly sampled IDs. The role-labelled topology contrast is negative in both subsets, with greater uncertainty in the smaller reused subset. Role-label contrasts remain small and uncertain. This check does not erase possible benchmark contamination, but it shows that the principal topology result is not driven only by the previously studied 200 task IDs.

Table 4. Exploratory direct-solver quality contrasts. Differences and intervals are percentage points; positive values favor D.

Contrast	Tasks	Difference	95% interval
D – SN	980	+0.27	[−0.88, 1.43]
D – SR	979	+0.95	[−0.17, 2.11]
D – MN	966	+4.59	[2.97, 6.18]
D – MR	970	+3.20	[1.82, 4.64]

Intervals resample tasks. These post hoc contrasts are outside the four-test Holm family and support estimation, not a new confirmatory family.

Table 5. Old/new task sensitivity for the four primary contrasts. Values are percentage points with task-bootstrap 95% intervals.

Contrast	Subset	Tasks	Difference [95% interval]
SR–SN	reused 200	194	−1.03 [−3.78, 1.72]
	new 800	788	−0.68 [−2.07, 0.72]
MR–MN	reused 200	190	+1.93 [−0.70, 4.56]
	new 800	774	+1.21 [−0.13, 2.58]
MN–SN	reused 200	192	−4.17 [−7.64, −0.87]
	new 800	776	−4.17 [−5.84, −2.58]
MR–SR	reused 200	190	−1.23 [−3.68, 1.23]
	new 800	781	−2.35 [−3.88, −0.81]

Every response was generated afresh. “Reused” means that the task ID had appeared in the earlier 200-task study; it is not reused output.

5.3 Resource accounting

Completed candidates use the full repeated inputs and exposed intermediate responses. Single-call role-labelled instructions have a valuation ratio of 0.939 relative to neutral instructions, or 6.08% lower API-equivalent valuation, on 997 complete three-repeat task pairs; their token ratio is 0.986. Within three calls, the corresponding valuation ratio is 0.978 and token ratio 0.990 on 979 task pairs. These label-associated resource differences are descriptive and smaller than those observed in the earlier mini study.

The topology effect is much larger. Three-call neutral instructions require 3.08 times the tokens and 2.67 times the valuation of single-call neutral instructions. With roles, the ratios are 3.09 and 2.78. Removing the cache discount leaves valuation ratios of 2.78 and 2.87. The values are API-equivalent valuations from measured counters, not subscription invoices or total research cost. Reasoning tokens are included in output and are not added twice.

For scale, the mean known totals per completed candidate are 3,972 tokens and \$0.000813 for D, 4,073 and \$0.000899 for SR, 4,131 and \$0.000957 for SN, 12,583 and \$0.002496 for MR, and 12,712 and \$0.002552 for MN. Aggregate known valuation over the five arms is \$23.07. Ratios remain the primary resource comparison because small completion-count differences make unpaired totals misleading.

For input count I , cached portion C , and output count O , the Luna valuation is $[0.20(I - C) + 0.02C + 1.20O]/10^6$ dollars at the base rates checked on 13 September 2026 [15, 18]. It values observed counters at a published API rate; it does not estimate provider overhead, researcher time, evaluator compute, cache-write surcharges, or total cost of ownership. The no-cache sensitivity

Table 6. Luna factorial resource ratios: first condition divided by second.

Contrast	Tasks	Tokens	Valuation	95% interval	No cache
SR – SN	997	0.986	0.939	[0.93, 0.95]	0.955
MR – MN	979	0.990	0.978	[0.97, 0.99]	0.986
MN – SN	983	3.078	2.672	[2.63, 2.71]	2.777
MR – SR	986	3.090	2.780	[2.74, 2.82]	2.867

Ratios divide sums of three-repeat task means on complete resource pairs. The displayed interval is for the API-equivalent valuation ratio. No cache removes its input-cache discount; subscription billing is not estimated.

Table 7. Earlier GPT-5.4-mini corroboration on the 200 reused task IDs. Differences and descriptive intervals are percentage points; n counts complete task pairs. The four exact McNemar tests form a Holm family.

Contrast	n	Δ	95% interval	Holm p
SR–SN	191	−1.05	[−4.71, +2.62]	1.000
MR–MN	193	+1.04	[−3.11, +5.70]	1.000
MN–SN	191	−4.71	[−9.42, 0.00]	.313
MR–SR	193	−2.07	[−6.22, +2.07]	1.000

uses $[0.20I + 1.20O]/10^6$ and tests whether the direction depends on discounted cached input. Every transmitted context and exposed preceding response is included, because that exposure is part of the treatment. Unknown counters are reported as unknown and are not zero-imputed.

5.4 Corroborating studies

An earlier GPT-5.4-mini factorial study used the same 200 task IDs later retained in the expanded allocation, but generated one fresh request per condition under the earlier model. Of 1,000 assignments, 996 completed; contrast eligibility is 191–193 task pairs. Its four effect directions match the Luna study (Table 7): labels are negative in one call and positive in three, while three calls are negative under both label settings. None rejects after Holm correction. The corroboration therefore concerns direction across two model aliases on the same 200 tasks, not an independent task sample or a second high-powered confirmation. The mini study’s three-call valuation ratios are 2.09 against SN and 2.50 against SR.

Other retained pilots vary instruction placement or change the reasoning procedure. Because they have different estimands, they are neither pooled nor invoked as support for the primary claim.

6 Self-Collaboration Comparison

We compare Self-Collaboration (SCC) with the single-call conditions using the same Luna model, medium reasoning and native evaluator. This comparison concerns complete workflows. The adapted author controller [4] begins with an analyst JSON plan, calls a developer for Python, and asks a tester to generate up to five example calls in a `check(candidate)` function. It executes the program and generated check; exception-free execution is its internal stopping signal, while printed values are not automatically compared with expected answers. The configured cap permits one initial implementation and at most one repair. The model receives the complete supplied task context and serialized role history. Reference solutions and benchmark tests remain external.

Table 8. SCC quality on observed matched repeats. Differences and intervals are percentage points; n counts contributing tasks.

Contrast	n	Δ	95% CI	Holm p
SCC–SR	883	–2.68	[–4.72, –0.60]	.0217
SCC–SN	882	–2.48	[–4.50, –0.43]	.0217

The original schedule contains 3,000 task-repeat blocks, each assigning fresh SR, fresh SN and SCC workflows. An amendment adopted after generation began, before inspecting SCC quality, fixes its first 2,000 randomized blocks (order seed 20260911). This gives 6,000 method assignments over 956 tasks: 212 tasks have one selected repeat, 444 have two, and 300 have three. Of these tasks, 941 pass reference controls. The unselected 3,000 method assignments are outside the amended denominator.

All 5,595 completed programs underwent native evaluation: 1,997 SR, 1,998 SN and 1,600 SCC. The comparison retains 397 infrastructure failures and eight historically paused attempts. Specifically, SCC has 396 Docker failures and four paused workflows; SR has three paused attempts; SN has one paused attempt and one stream failure. No submitted attempt was regenerated. Quality is observed for 1,964 SR repeats, 1,965 SN repeats and 1,574 SCC repeats after reference controls, with 1,067, 1,064 and 809 passes, respectively. These are observed-repeat counts, not independent tasks or best-of-three rates.

For each comparator, average SCC minus comparator outcomes over matching selected repeats with both outcomes observed, then average equally over eligible tasks with at least one such pair. Thus a task with three selected repeats has the same weight as a task with one. Two-sided task-level t -tests for SCC–SR and SCC–SN form a separate two-test Holm family. Intervals use 10,000 whole-task PCG64 draws, seed 20260911 and linear percentile endpoints.

Both conditional contrasts are negative (Table 8). The source-group sensitivity intervals are similar: [–4.72, –0.58] points against SR and [–4.51, –0.43] against SN for the union partition. However, fixed-denominator extreme bounds across all 941 eligible selected tasks are [–12.98, +6.61] and [–13.14, +6.43] points. They allow either sign. The observed-pair results therefore do not establish SCC inferiority over the full assigned population.

Resource comparisons require both complete workflows with valid counters. The ratio of paired task means for API-equivalent valuation is 3.138 against SR (896 tasks; 95% interval [3.067, 3.208]) and 2.939 against SN (895 tasks; [2.871, 3.010]). These ratio-of-means intervals use 10,000 whole-task draws with seed 20260914. They concern the observed complete workflows and do not combine quality preservation with a monetary advantage. Availability and matched effects before and after the amendment are retained as descriptive checks; they do not identify a causal time effect.

A separate diagnostic evaluates all 399 retained developer programs from incomplete SCC workflows without restarting the controller or appending its generated checks. It finds 196 passes among 392 control-eligible programs; seven programs are control-ineligible. These intermediate-program results do not replace any unknown whole-workflow outcome in the comparison.

7 Discussion

The completed factorial study estimates a conditional difference between the tested workflows. In the full-context, no-compression protocol, distributing planning, implementation, and critique across three fresh calls increases repeated transmission and measured output, and has lower observed-pair native test success than keeping the same operations in one response. The result is a property of the tested call structure, model, task population, deadline, and evaluation contract. It

does not show that separate agents are generally ineffective or that additional interaction cannot help when agents have tools, retrieval, repository state, or a matched computational budget.

The role-label intervention is smaller and uncertain. The three-call role estimate is descriptively positive, but it does not pass the corrected primary test. The evidence therefore does not support a universal quality benefit from naming internal functions, nor does failure to reject support quality equivalence. Internal labels can still organize a prompt or communicate a useful decomposition; the experiment measures whether they changed this benchmark outcome under identical operation sentences.

Resource savings must be read together with quality. Single-call role labels slightly reduce measured valuation in this study, while three calls cost roughly 2.7–2.8 times as much. Because there is no hard matched output budget, the accounting includes the induced reasoning length, repeated context, intermediate responses, and cache state. No joint quality-cost superiority claim is made. The no-cache sensitivity changes the magnitude but preserves the large topology contrast.

The external evaluator also prevents a workflow from receiving credit for a persuasive self-reported correction that did not change executable behavior. Only the final candidate enters the native tests. The direct condition has the lowest observed resource means. Its estimated quality differences from the one-call staged conditions are small and their intervals include zero; these exploratory tests were not designed to establish equivalence. It has higher observed paired success than both three-call conditions, although those comparisons were not preregistered as a confirmatory family.

For fixed-context function synthesis without tool feedback, the evidence supports a direct or one-call staged workflow as the default and requires an explicit source of value—parallel search, private state, tools, retrieval, or executable feedback—before paying for additional calls. Evaluations should report both task-paired executable quality and provider-independent token counts.

Three repeats improve the estimate for sampled tasks but do not make 15,000 independent engineering problems. The bootstrap keeps repeats together, and each task contributes an equal-weighted per-generation mean. This avoids treating repeated requests as new task units or using best-of-three selection. A separate-call workflow could still help when tools, retrieval, repository state, parallel subtasks, recovery, or a matched compute budget are available; those capabilities are outside this fixed-context isolation.

The earlier studies narrow robustness: the mini factorial uses a different model and smaller task sample, the layout study changes instruction placement, and the INoT adaptation changes the algorithmic procedure. Their results remain separate from the Luna quality tests. Repeated outcomes improve estimates for sampled tasks but do not create independent tasks or remove benchmark training contamination.

8 Limitations and Reproducibility

The work uses two model aliases, each at one reasoning setting and one client version, and does not identify model-family effects. The smaller mini study reuses the same 200 task IDs and has only one request per condition; its directional agreement is corroboration rather than independent replication. Provider weights, temperature, and sampling seeds are not immutable or fully exposed. The expanded sample reuses those 200 task IDs, so it is not 1,000 newly held-out tasks. Public benchmark availability leaves training contamination unresolved, and related tasks may reduce effective independence.

Unavailable generation and infrastructure failures retain unknown quality. Observed test failures and benchmark timeouts count as unsuccessful programs on eligible tasks. Continuation allowed only previously untouched assignments after transport interruptions; started calls were not retried. Available-case estimates concern the observed eligible subset. Extreme assignment bounds show

what missing binary endpoints permit but cannot remove non-random missingness. The study therefore avoids claims of equivalence, non-inferiority, or an economically optimal design.

Native tests provide executable evidence but inherit the benchmark’s specification-test relationship. Reference and incorrect controls detect unusable environments and insensitive negative checks; they do not prove that every assertion captures the natural-language task. The fixed-context protocol excludes retrieval, tools, repository interaction, and adaptive test feedback. Consequently, the results do not establish performance on repository repair or end-to-end software development.

The intervention is narrower than the broader prototype that motivated it: context selection, selective reruns, an auxiliary checker, and compression are disabled. This isolates the staged-generation core but cannot substitute for a full deployment evaluation. SCC has a further boundary because its generated checks and exception-free stopping signal define a different feedback controller. Its higher valuation and lower observed-pair success concern that whole workflow; they do not isolate the contribution of role labels within SCC. Substantial SCC infrastructure missingness limits population-level conclusions.

Task IDs, prompts, responses, statuses, extraction flags, counters, controls, native reports, and analysis seeds are retained in the project records. Reconstruction therefore includes assignment denominators and paired subsets. A new provider revision, dependency version, or task-test interpretation could change the rates. Reproduction should preserve the disclosed alias, client, context bytes, deadlines, control gate, and missingness rules and report any deviation.

9 Conclusion

Across 15,000 assigned Luna generations, role labels do not pass the corrected quality tests. Splitting the same full-context operations across three calls lowers paired executable success by 2.13–4.17 percentage points and raises API-equivalent valuation by factors of 2.67–2.78. The negative topology direction also appears in the smaller mini study and persists in the 800 newly sampled Luna task IDs. The direct solver has small, inconclusive differences from the one-call staged workflows and higher observed paired success than both three-call workflows in exploratory comparisons. The 2,000-block SCC comparison also shows higher valuation and lower observed-pair success than fresh single-call controls, but missing outcomes prevent a full-population quality ranking. Named roles and extra calls should therefore be treated as separate design choices and justified with executable evidence rather than assumed to provide agentic benefit.

Data Availability

An anonymized replication package accompanies the submission as supplementary material. It contains the frozen protocols and amendments, task selections, assignment-level outcomes, status and usage counters, primary candidate programs, all primary native reports, control evidence, pinned environment specifications, the benchmark evaluator source and data under their original license, source-dependence partitions, analysis scripts, exact table inputs, and a manifest of file hashes. It can rerun the primary, mini, SCC, and source-component numerical analyses; checkpoint the headline table values; and verify that the primary programs and native statuses exactly match the immutable evaluator records without provider credentials. Full conversational response bodies, SCC candidate source text, and operational transport traces are withheld because they are not needed for these checks and may contain incidental generated identifiers or machine-local metadata. Identity-bearing repository history, local paths, account identifiers, and credentials are excluded. The retained complete archive will be released after the double-anonymous review period, subject to benchmark and provider terms.

References

- [1] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 143–153. doi:10.1145/3359591.3359735
- [2] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ktrw68Cmu9c>
- [3] Junhao Chen, Xiang Li, Yibin Xu, Yuehan Cui, Fangsheng Weng, Hao Zhao, Fei Ma, and Qi Tian. 2026. PairCoder: Pair Programming-Inspired Two-Agent Collaboration for Code Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 3043–3058. doi:10.18653/v1/2026.findings-acl.149
- [4] Yihong Dong. 2024. *Self-Collaboration Code Generation: Accompanying Python Software*. Documented author implementation associated with Dong et al.; analyst–coder–tester session with generated-check execution. Historical 2024 analyst–coder–tester implementation, associated with arXiv:2304.07590; workflow and transport adaptations are specified in the text.. <https://github.com/YihongDong/Self-collaboration-Code-Generation>
- [5] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. Preprint: arXiv:2304.07590. doi:10.1145/3672459
- [6] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003
- [7] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [8] Yunsong Hong. 2026. Learning How Much to Collaborate: Difficulty-Aware Topology Selection for Multi-Agent Code Generation. arXiv:2609.13890 <https://arxiv.org/abs/2609.13890>
- [9] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 doi:10.48550/arXiv.2312.13010
- [10] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 4912–4944. doi:10.18653/v1/2024.acl-long.269
- [11] Sayash Kapoor, Benedikt Strobel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=Zy4uFzMviZ>
- [12] Pedro Henrique Luz de Araujo, Paul Röttger, Dirk Hovy, and Benjamin Roth. 2025. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 26857–26886. doi:10.18653/v1/2025.emnlp-main.1364
- [13] James G. MacKinnon, Morten Ørregaard Nielsen, and Matthew D. Webb. 2023. Cluster-Robust Inference: A Guide to Empirical Practice. *Journal of Econometrics* 232, 2 (2023), 272–299. doi:10.1016/j.jeconom.2022.04.001
- [14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2019
- [15] OpenAI. 2026. *API Pricing*. Standard text-token rates. <https://developers.openai.com/api/docs/pricing>
- [16] OpenAI. 2026. *Codex Non-Interactive Mode and JSONL Event Stream*. <https://developers.openai.com/codex/noninteractive>
- [17] OpenAI. 2026. *GPT-5.4 Mini Model Documentation*. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>
- [18] OpenAI. 2026. *GPT-5.6 Luna Model Documentation and Pricing*. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
- [19] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-0377
- [21] Haoran Sun and Shaoning Zeng. 2025. Introspection of Thought Helps AI Agents. arXiv:2507.08664 <https://arxiv.org/abs/2507.08664>

- [22] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460 <https://arxiv.org/abs/2604.02460>
- [23] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. doi:10.1145/3715754
- [24] Jiawei Xu, Arief Koedwidy, Sisong Bei, Yan Han, Baixiang Huang, Dakuo Wang, Yutong Chen, Zheshen Wang, Peihao Wang, Pan Li, and Ying Ding. 2026. Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline. arXiv:2601.12307 <https://arxiv.org/abs/2601.12307>
- [25] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Published as a NeurIPS 2024 paper; arXiv version retained for stable metadata. arXiv:2405.15793 <https://arxiv.org/abs/2405.15793>
- [26] Huan Zhang, Wei Cheng, Yuhan Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1319–1331. doi:10.1145/3691620.3695506
- [27] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 15126–15154. doi:10.18653/v1/2024.findings-emnlp.888
- [28] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-Step. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 851–870. doi:10.18653/v1/2024.findings-acl.49
- [29] Yueheng Zhu, Chao Liu, Xuan He, Xiaoxue Ren, Zhongxin Liu, Ruwei Pan, and Hongyu Zhang. 2026. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation. *IEEE Transactions on Software Engineering* 52, 2 (2026), 631–650. doi:10.1109/TSE.2025.3642621
- [30] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations*. Published as a conference paper at ICLR 2025. <https://arxiv.org/abs/2406.15877>